

Algorísmia FIB UPC Q1 2024–2025

Examen Final

10 de gener de 2025 (8:00)

Durada: 2 h 50 min

[Exercici 1] (2 punts): Preguntes curtes

- (a) **(0.5 punts)** Donat un graf unicíclic G , és a dir, G connex i contenint un únic cicle, amb pesos reals a les arestes, dissenyeu un algorisme el més eficient que pugueu per determinar el nombre d'arbres d'expansió de cost mínim del graf G . Justifiqueu la correctesa i el cost de l'algorisme proposat.

Una solució Como el grafo es conexo y unicíclico obtenemos un árbol de expansión al eliminar una arista del ciclo. Para que el árbol de expansión tenga coste mínimo, de acuerdo con la regla roja, la arista eliminada tiene que tener peso máximo entre las del ciclo. Por lo tanto, el número de MSTs es el de aristas del ciclo con peso máximo en él. Como algoritmo utilizo un BFS para identificar el ciclo. Después hago un recorrido del ciclo calculando el peso de la arista de mayor peso en él y el número de aristas con este peso. El coste total es $O(n)$ ya que un grafo unicíclico conexo tiene n aristas.

Decidiu i justifiqueu si els enunciats següents són certs o falsos.

- (b) **(0.5 punts)** Donada una matriu de n enters, cadascun pertanyent a $\{-10, \dots, 0, \dots, 20\}$, podem ordenar la matriu en $O(n)$ temps en cas pitjor.

Una solució Cierto. El rango de los valores está acotado (hay 31 valores diferentes), podemos usar una modificación de counting sort utilizando un vector en el que contamos el número de apariciones de cada valor posible. El coste de esta ordenación es $O(n+31) = O(n)$.

Nota No se ha valorado la propuesta de ordenación utilizando radix salvo en aquellos casos en que se ha especificado claramente una modificación que permite ordenar sin usar valores negativos.

- (c) **(0.5 punts)** Considereu un graf dirigit $G = (V, E)$ amb dues assignacions de pes positives w_1 i w_2 , on, per a qualsevol $e \in E$, $w_2(e) = w_1(e)^2$. Llavors, per a dos vèrtexs qualsevol $u, v \in V$, qualsevol camí més curt entre u i v a (V, E, w_1) també és un camí més curt a (V, E, w_2) .

Una solució Falso. Si consideramos un grafo con tres vértices a, b, c y aristas formando un triángulo con pesos $w_1(a, b) = w_2(b, c) = 1$ y $w(a, c) = 2$. Los dos caminos de a a c son de distancia mínima bajo w_1 . $w_2(a \rightarrow b \rightarrow c) = 2$ pero $w_2(a \rightarrow c) = 4$ con lo que $a \rightarrow c$ deja de ser camino mínimo bajo w_2 .

- (d) **(0.5 punts)** Si a una xarxa de flux totes les capacitats són nombres racionals aleshores el valor del flux màxim també és un nombre racional.

Una solució Cierto. Hemos visto que el valor del flujo de valor máximo coincide con la capacidad del corte de capacidad mínima. Las capacidades de las aristas son números racionales, por lo que la suma de capacidades del corte con capacidad mínima es la suma de números racionales que es un número racional.

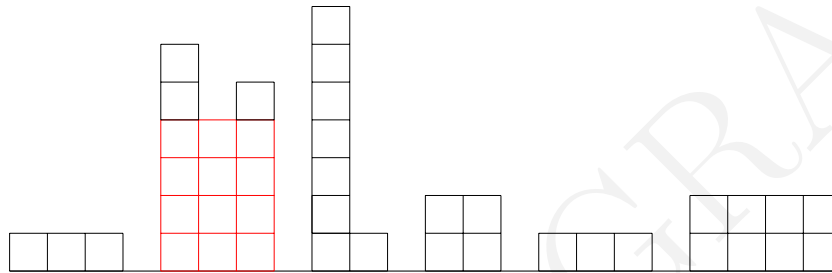
Nota Se ha penalizado especialmente el error de argumentas que un número entero no es un número racional.

[Exercici 2] (3 punts): Volem ajudar al professor Valiente amb el disseny de la forma d'un panel nou per l'entrada del departament de Ciències de la Computació. El professor us dona n columnes, totes de la mateixa amplada, però amb alçades diferents: $h_1, h_2, \dots, h_n$. Les columnes poden reordenar-se a sobre d'una línia base per a definir diferent formes.

El director de CS vol penjar al panel un cartell rectangular.

Si a algun panel hi ha j columnes consecutives amb alçada k o més, podem penjar un rectangle de mides $j \times k$.

A l'exemple de sota tenim un panel amb 3 columnes consecutives amb alçada 4 o més, llavors podem penjar un rectangle d'àrea 12 a sobre d'aquestes 3 columnes.



- Doneu un algorisme polinòmic per trobar l'àrea més gran per poder penjar un cartell quan les columnes segueixen un ordre inicial donat $h_1, h_2, \dots, h_n$, és a dir, quan les columnes no es poden moure.
- Doneu un algorisme polinòmic (i eficient) per reordenar les columnes de manera que l'ordenació proporcioni el panel que maximitzi l'àrea per penjar el cartell.

Una solució bàsica (Apartat a)

L'àrea més gran (cartell rectangular més gran) estarà limitada en alçada per la columna més baixa dins del rectangle. Per tant, l'algorisme més senzill que podem proposar calcula, per a cada columna c_i , les columnes límit a l'esquerra c_l i a la dreta c_r fins on es pot estendre un cartell d'alçada h_i . És a dir, trobar c_l i c_r ($l \leq i \leq r$) de manera que $h_j \geq h_i$ ($l \leq j \leq r$), però $h_{c_l-1} < h_i$ i $h_{c_r+1} < h_i$. La correcció de l'algorisme es garanteix atès que per a cada columna c_i l'algorisme identifica correctament els límits màxims d'extensió i, per tant, l'àrea més gran del cartell que es pot col·locar limitada per cada columna. L'algorisme resultant té una complexitat $O(n^2)$ temporal.

Una solució eficient (Apartat a)

Podem plantejar una solució golafre més eficient que per a totes les columnes c_i calculi les seves corresponents columnes límit (a l'esquerra i a la dreta) en dos vectors ($left[i]$ i $right[i]$) en temps lineal (n).

Sense pèrdua de generalitat, per calcular les components del vector $left$, recorrerem les columnes d'esquerra a dreta. Per calcular cada $left[i]$, volem trobar el primer h_j a l'esquerra de h_i tal que $h_j < h_i$. Observeu que, donades dues columnes adjacents h_{i-2} i h_{i-1} tal que $h_{i-2} \geq h_{i-1}$, si h_i es pot estendre a h_{i-1} , també es pot estendre a h_{i-2} . Per tant, és treball redundant comparar contra columnes adjacents monòtonament no creixents.

Explotarem aquesta propietat fent servir una pila que guardarà les columnes en ordre d'altures estrictament creixents mentre les recorrem d'esquerra a dreta. Per a cada columna c_i , compararem la seva alçada $h[i]$ amb l'alçada de la columna emmagatzemada a la part superior de la pila ($h[\text{stack.top}()]$). Si $h[\text{stack.top}()] \geq h[i]$, la columna a $\text{stack.top}()$ no serveix com a límit per a $h[i]$ perquè és més alta o igual. Traurem elements de la pila fins que $\text{stack.top}()$ sigui més baixa que $h[i]$ o fins que la pila quedi buida. A continuació, presentem una proposta d'algorisme.

```
function LARGEST_RECTANGLE_AREA( $h_1, \dots, h_n$ )
   $left[1 \dots n] \leftarrow 0$ 
   $right[1 \dots n] \leftarrow 0$ 
   $stack \leftarrow \emptyset$ 
  ▷ Compute left limits
  for  $i \leftarrow 1$  to  $n$  do
    while  $stack \neq \emptyset$  and  $h[\text{stack.top}()] \geq h[i]$  do
       $stack.pop()$ 
    end while
     $left[i] \leftarrow \text{stack.top}()$  if  $stack \neq \emptyset$  else  $0$ 
     $stack.push(i)$ 
  end for
   $stack \leftarrow \emptyset$ 
  ▷ Compute right limits
  for  $i \leftarrow n$  downto  $1$  do
    while  $stack \neq \emptyset$  and  $h[\text{stack.top}()] \geq h[i]$  do
       $stack.pop()$ 
    end while
     $right[i] \leftarrow \text{stack.top}()$  if  $stack \neq \emptyset$  else  $n + 1$ 
     $stack.push(i)$ 
  end for
  ▷ Calculate maximum area
   $max\_area \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
     $width \leftarrow right[i] - left[i] - 1$ 
     $area \leftarrow h[i] \times width$ 
     $max\_area \leftarrow \max\{max\_area, area\}$ 
  end for
  return  $max\_area$ 
end function
```

Atès que cada índex de columna només pot entrar i sortir una vegada de la pila, els tres bucles tenen cost lineal. Per tant, l'algorisme resultant té complexitat $O(n)$ temporal i espacial.

Una solució eficient (Apartat b)

Si es permet reorganitzar les columnes, ordenar-les per altures garanteix l'àrea potencial més gran, ja que les columnes més altes contribueixen a un rectangle més ample. Per fer-ho, ordenarem les altures en ordre no creixent (de la més alta a la més baixa). Una vegada ordenades, calcularem l'àrea del rectangle més gran utilitzant les altures ordenades.

```

function MAXIMIZEAREABYREORDERING( $h_1, \dots, h_n$ )
   $h \leftarrow \text{MERGESORT}(h)$   $\triangleright O(n \log(n))$  Sort heights in descending order
   $max\_area \leftarrow 0$ 
  for  $i \leftarrow 1$  to  $n$  do
     $area \leftarrow h[i] \times i$   $\triangleright$  Width increases as we add more columns
     $max\_area \leftarrow \max\{max\_area, area\}$ 
  end for
  return  $max\_area$ 
end function

```

Un argument d'intercanvi pot demostrar la correcció de la nostra estratègia golafre i que aquesta genera la solució òptima. Observeu que si prenem el resultat de la solució anteriorment proposada i intercanviem dues columnes h_i i h_j , succeiria que (1) l'àrea més gran seguiria sent la mateixa o (2) seria més petita atès que les columnes més altes estarien més lluny del principi, cosa que implicaria que es perdria la possibilitat de formar un rectangle més gran en reduir l'alçada dominant (la menys alta) per a un rang més gran de columnes consecutives. Per tant, qualsevol intercanvi que trenqui aquest ordre no pot millorar la solució, cosa que demostra que l'estratègia golafre és òptima.

Pel que fa al cost de l'algorisme, podem ordenar utilitzant un algorisme d'ordenació $O(n \log(n))$, com el mergesort i, per tant, l'algorisme té cost temporal $O(n + n \log(n)) = O(n \log(n))$.

[Exercici 3] (3 punts):

Donada una seqüència $A = (a_1, a_2, a_3, \dots, a_n)$ de n nombres enters positius, si s'escriuen els enters en l'ordre en què apareixen a A i s'insereixen els símbols $+$ o $-$ davant de cada enter, obtindríem una expressió aritmètica. Si l'avaluació d'una expressió aritmètica d'aquest estil dona valor v , diem que v està *calculat* o que el valor v és *calculable* per la seqüència A .

Per exemple, si es considera la seqüència $(7, 12, 1, 9, 5)$ de cinc nombres enters positius, tenim:

$$+7 - 12 - 1 + 9 + 5 = 8,$$

$$-7 - 12 + 1 - 9 + 5 = -22,$$

$$-7 + 12 - 1 - 9 + 5 = 0,$$

és a dir, els enters 8, -22 , 0 són calculables per aquesta seqüència.

Donada una seqüència $A = (a_1, a_2, a_3, \dots, a_n)$, sigui $S = a_1 + a_2 + a_3 + \dots + a_n$, i sigui $T \in \mathbb{Z}$. Llavors, perquè T sigui calculable per A , ha de passar que $-S \leq T \leq S$. No obstant això, aquesta condició només és necessària (però no suficient). En l'exemple de cinc elements anterior, $7 + 12 + 1 + 9 + 5 = 34$, però un enter senar com 23 o un enter parell com 30, malgrat estar dins l'interval $[-34, 34]$, no poden ser calculats per aquesta seqüència.

Donats una seqüència $A = (a_1, a_2, a_3, \dots, a_n)$ i un enter T , dissenyeu un algorisme de programació dinàmica (tan eficient com pugueu) per a **determinar si T és calculable per la seqüència A donada**, i si ho és, **com**. Podeu assumir que $-S \leq T \leq S$.

Observació: Noteu que pot haver-hi múltiples maneres de calcular el mateix valor T des d'una seqüència donada A . Només cal que en doneu una.

Una solució:

Dada una secuencia $A = (a_1, a_2, a_3, \dots, a_n)$ y un entero T , tenemos que decir si T es o no *calculable* a partir d' A . Por lo tanto, es un problema decisional que requiere una respuesta booleana.

Si T es calculable, tenemos una expresión válida que nos permite calcular T . En esta expresión, si miramos el último valor tenemos que o bien a_n aparece con signo $+$ o con signo $-$. En el primer caso, $T - a_n$ es calculable a partir de $(a_1, a_2, a_3, \dots, a_{n-1})$ y en el segundo $T + a_n$ es calculable a partir de $(a_1, a_2, a_3, \dots, a_{n-1})$. Y al inversa, si $T - a_n$ o $T + a_n$ son calculables a partir de $(a_1, a_2, a_3, \dots, a_{n-1})$, tenemos que T es calculable a partir de A .

A la vista de esta caracterización recursiva, considero las subsecuencias $A_i = (a_1, \dots, a_i)$, $1 \leq i \leq n$, y los valores x , $-S \leq x \leq S$. Y, para cada combinación, el valor booleano $C(i, x)$ que indica si x es calculable a partir de A_i o no.

De acuerdo con el análisis anterior, tenemos la siguiente reurrencia:

$$C(i, x) = \begin{cases} x = a_1 \text{ or } x = -a_1 & \text{si } i = 1 \\ C(i-1, x - a_i) \text{ or } C(i-1, x + a_i) & \text{si } i > 1 \end{cases}$$

El caso base, se corresponde con un secuencia de un único elemento y contempla los dos únicos valores calculables a partir de él.

La solución al problema la representa el valor $C(n, T)$.

Para implementar esta recurrencia podemos utilizar una matriz de dimensiones $(n+1) \times (2S+1)$ a rellenar por filas, con coste constante por elemento. Lo que nos da un algoritmo con coste temporal y espacial $O(nS)$.

Reconstrucción de la solución:

Para reconstruir la solución es suficiente hacer un recorrido empezando por $C(n, T)$. Unos de sus predecesores $C(n-1, T-a_n)$ o $C(n-1, T+a_n)$ es cierto. Seleccionamos un predecesor cierto, este determinará el signo de a_n y pasará a ser el nuevo elemento del recorrido. En el caso general llegaremos a un elemento $C(i, x)$ tras haber determinado los signos de los elementos $a_{i+1}, \dots, a_n$. Procederemos de la misma manera hasta completar la asignación de signos. El coste total de esta reconstrucción es $O(n)$.

[Exercici 4] (2 punts):

Tenim dos jugadors P_0 i P_1 i dues llistes, X i Y . La llista X conté els noms d' n actrius amb les pel·lícules on han participat, i la llista Y conté els noms d' n actors i les pel·lícules on han participat. Cada jugador disposa de còpies de les llistes X i Y amb la informació completa de qui ha aparegut en una pel·lícula amb qui.

El joc consisteix en el següent, el jugador P_0 diu el nom d'una actriu $x_1 \in X$, aleshores P_1 ha de donar el nom d'un actor $y_1 \in Y$ que ha aparegut a una pel·lícula amb x_1 , a continuació P_0 ha de donar el nom d'una actriu $x_2 \in X$, $x_2 \neq x_1$, que ha aparegut a una pel·lícula amb y_1 , i així successivament fins que un dels dos jugadors no pot donar cap nom diferent. Noteu que la restricció important és que no es poden repetir noms al llarg de la partida. El jugador que per primer cop no pot donar cap nom és el jugador que perd.

Podeu observar que a una partida P_0 i P_1 generen col·lectivament una seqüència

$$x_1, y_1, x_2, y_2, \dots, x_i, y_i, \dots,$$

de manera que cada actor/actriu en la seqüència ha coprotagonitzat al costat de l'actriu/actor immediatament anterior, i no es repeteixen noms.

Una estratègia per al jugador P_0 és un algorisme que pren com a entrada una seqüència $x_1, y_1, x_2, y_2, \dots, x_i, y_i$ i genera un x_{i+1} legal, si això és possible. De la mateixa manera podem definir una estratègia per a P_1 .

- Doneu una formalització de les propietats que es compleixin quan una partida és guanyadora per a P_0 i el mateix quan ho és per a P_1 .
- Dissenyeu un algorisme de temps polinòmic per a decidir si un dels dos jugadors pot forçar una estratègia on ell guanyi. Ajuda: penseu com farien servir els jugadors un *matching* entre actrius i actors en dissenyar les seves respectives estratègies.

Una solució:

Para analizar las estrategias del juego y formalizar las propiedades que nos piden considero el grafo bipartido $G = (X, Y, E)$ en el que tenemos un vértice por cada actriz (X) y un vértice por cada actor (Y) y una arista entre actriz y actor si han trabajado juntos en alguna película.

Una partida ganadora para P_0 es una secuencia $x_1, y_1, x_2, y_2, \dots, x_i, y_i$ representa una partida ganadora para P_1 si y solo si $x_1, y_1, x_2, y_2, \dots, x_i, y_i$ es un camino simple en G tal que y_i no tiene ningún vecino fuera de $x_1, \dots, x_i$. Notemos también que la secuencia tiene longitud impar.

Una partida ganadora para P_1 es una secuencia $x_1, y_1, x_2, y_2, \dots, x_i$ representa una partida ganadora para P_0 si y solo si $x_1, y_1, x_2, y_2, \dots, x_i$ es un camino simple en G tal que x_i no tiene ningún vecino fuera de $y_1, \dots, y_{i-1}$. Notemos también que la secuencia tiene longitud impar.

Siguiendo la ayuda, podemos ver que en una partida ganadora para P_0 , $x_1, y_1, x_2, y_2, \dots, x_i, y_i$, aparecen dos matchings entrelazados: $(x_1, y_1), \dots, (x_i, y_i)$ determinado por las respuestas de P_1

y $(y_1, x_2), \dots (y_{n-1}, x_n)$ determinado por las respuestas de P_0 . Notemos que los dos matchings tiene el mismo número de aristas.

En una partida ganadora para P_1 , $x_1, y_1, x_2, y_2, \dots, x_i$, aparecen dos matchings entrelazados: $(x_1, y_1), \dots (x_{i-1}, y_{i-1})$ determinado por las respuestas de P_1 y $(y_1, x_2), \dots (y_{i-1}, x_i)$ determinado por las respuestas de P_0 . Notemos que en este caso el segundo matching tiene una arista menos que el primero.

Parece que una forma de forzar que P_1 gane (independientemente de lo que haga P_0) es cuando podemos garantizar que no hay un matching para P_0 de longitud mayor que el usado por P_1 . Esto pasa cuando el grafo tiene un matching perfecto.

Si G tiene un matching perfecto M y P_1 responde siempre de acuerdo con la pareja establecida por M , P_1 gana, ya que sea la que sea la actriz que proponga P_0 , siempre tiene actor emparejado con ella en M que $P - 1$ todavía no a propuesto.

Si G no tiene un matching perfecto, considero un matching M de cardinalidad máxima. Como en M no es perfecto al menos hay una actriz que no está emparejada en M . Si P_0 empieza con una actriz no emparejada en M y responde siempre de acuerdo con M ganará independientemente de lo que haga P_1 . Para que P_1 gane tendría que encontrar una manera de solapar con el matching M formando un camino que empiece en X en un actriz fuera de M y acabe en Y en un actor fuera de M , pero entonces podríamos ampliar M en al menos una arista y no sería un matching de tamaño máximo.

A la vista de este análisis si G tiene un matching perfecto P_1 tiene una estrategia ganadora y en caso contrario P_0 tiene una estrategia ganadora.

Para obtener un matching the tamaño máximo en un grafo bipartido hemos visto en clase un algoritmo basado en flujo con coste $O(n(n+m))$. Dependiendo de si es perfecto o no podemos decidir cual de los dos jugadores tiene una estrategia ganadora y la estrategia que le llevará a ganar.